TECHNICAL PROJECT DESCRIPTION

College0

AI-Enabled Online College Program Management System

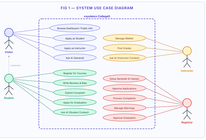
A full-stack web application for managing a graduate-level college program with AI-powered assistance, semester-lifecycle management, automated rule enforcement, and role-based access control.

React + TypeScript · Supabase (PostgreSQL) · Supabase Auth + RLS · Supabase Edge Functions · pgvector (RAG) · OpenAI GPT-4o · Tailwind CSS · Vercel

SYSTEM TYPE Web-Based SPA (React + Vite)	BACKEND / DBAS Supabase (PostgreSQL 15)
AI FEATURE RAG pipeline + OpenAI GPT-4o	VECTOR STORE pgvector via Supabase
AUTHENTICATION Supabase Auth (JWT + RLS)	CODEBASE SCALE ~10 students (showcase system)
USER ROLES Registrar · Instructor · Student · Visitor	REPORT DATE May 2026

Use Case Diagram & Functional Requirements

FIG 1 — SYSTEM USE CASE DIAGRAM



Functional Requirements Mapping

#	Use Case / Feature	Actor(s)	Business Rule	Status	Implementation
F01	Public Dashboard (Application cases, top GPA students)	All / Visitor	Always visible; no auth required	✓ Done	React + Supabase RLS public view
F02	Student Application & Approval	Visitor / Registrar	GPA >3.0 & quota not full → register should accept; override requires justification	✓ Done	applications table + Edge Function
F03	Instructor Application & Approval	Visitor / Registrar	Registrar approves/rejects freely; no justification needed; approved → assigned classes	✓ Done	Same applications flow, role-instructor
F04	First Login Password Change + New Student Tutorial	Student	System password must be changed on first login; tutorial shown to new students	✓ Done	Supabase Auth + profiles.first_login flag
F05	Semester Phase Management (4 phases)	Registrar	Setup → Registration → Running → Grading; strict phase gating enforced server-side	✓ Done	semesters table + Edge Function transitions
F06	Course Registration (2–4 courses, no time conflicts)	Student	No time conflict; upper limit check; waitlist if full; retake allowed if prior F	✓ Done	enrollments table + Postgres constraint fn
F07	Waitlist Management	Instructor	Only the course instructor can admit students from waitlist	✓ Done	enrollments.status + 'waitlisted'
F08	Course Cancellation + Special Registration Period	System / Student	<3 students → cancel course; affected students get one extra registration window	✓ Done	Edge Function triggered at phase start
F09	Class Reviews & Star Ratings (1-5)	Student	Anonymous to non registrars; taboo filter; rating disabled after grades posted	✓ Done	reviews table + RLS + taboo_words table
F10	Warning System (students & instructors)	System / Registrar	Auto-warnings: <2 courses enrolled, low rating <2, grade posted late, taboo hit; manual warnings	✓ Done	warnings table + Edge Function
F11	Suspension, Termination & Honor Roll Logic	System	3 warnings → suspend; GPA <2 or double F → terminate; honor roll requires 4 warning-free	✓ Done	profiles.status + Postgres triggers
F12	Complaint System (student → instructor, student → instructor)	Student / Instructor	Registrar must act on all escalations; unresolved complaint forces registrar decision	✓ Done	complaints table + admin action log
F13	Graduation Application	Student / Registrar	8 courses completed; registrar verifies required courses; warning if premature	✓ Done	graduation_applications + course check fn
F14	AI Chatbot — RAG + LLM Fallback	All roles	Role-scoped context; vector search first; GPT-4o fallback with hallucination warning shown	✓ Done	pgvector + OpenAI API via Edge Function
F15	Creative Feature: AI Smart Study Groups	Student	AI suggests study partners from co-enrolled students based on schedule and profile	✓ Done	study_groups table + GPT-4o Edge Function

Database Schema — Supabase (PostgreSQL 15 + pgvector)

All tables live in the `public` schema of a Supabase project. Row Level Security (RLS) is **enabled on every table**. Authentication is via `supabase.auth.users`; the `profiles` table extends it with application data. Foreign keys use `ON DELETE CASCADE` where appropriate. All tables include an `updated_at timestamptz` field maintained by a shared Postgres trigger.

profiles

id	uuid PK → auth.users
role	enum(registrar,instructor,student,visitor)
full_name	text not null
student_id	text unique nullable
status	enum(active,suspended,terminated,graduated)
warning_count	int default 0
cumulative_gpa	numeric(3,2)
honor_roll_count	int default 0
first_login	bool default true

semesters

id	uuid PK
name	text (e.g. 'Fall 2026')
phase	enum(setup,registration,running,grading,closed)
quota	int
created_by	uuid → profiles
created_at	timestamptz

classes

id	uuid PK
semester_id	uuid → semesters
course_code	text
title	text
instructor_id	uuid → profiles
schedule_time	tstzrange
max_students	int
avg_rating	numeric(3,2) computed
is_cancelled	bool default false

enrollments

id	uuid PK
student_id	uuid → profiles
class_id	uuid → classes
status	enum(enrolled,waitlisted,dropped)
grade	enum(A,A-,B+,B,B-,F) nullable
enrolled_at	timestamptz
UNIQUE	(student_id, class_id)

applications

id	uuid PK
applicant_email	text
role_requested	enum(student,instructor)
prior_gpa	numeric(3,2) nullable
status	enum(pending,accepted,rejected)
rejection_reason	text nullable
reviewed_by	uuid → profiles

reviews

id	uuid PK
class_id	uuid → classes
author_id	uuid → profiles
stars	int check(1 to 5)
body	text (profanity)
filtered_body	text (taboo → *)
is_hidden	bool default false

warnings

id	uuid PK
target_id	uuid → profiles
reason	text
issued_by	uuid → profiles or system
semester_id	uuid → semesters
is_removed	bool default false (honor roll)

complaints

id	uuid PK
filed_by	uuid → profiles
against	uuid → profiles
description	text
resolution	text nullable
status	enum(open,resolved)

document_embeddings (RAG)

id	uuid PK
content	text (course chunk)
embedding	vector(1536) pgvector
metadata	jsonb {category, scope, title}
created_at	timestamptz

taboo_words

id	uuid PK
word	text unique
added_by	uuid → profiles (registrar)
created_at	timestamptz

Additional tables: `graduation_applications` (id, student_id, semester_id, status, notes); `study_groups` (id, class_id, members jsonb, ai_suggested bool, created_at). All tables have `updated_at` maintained via shared trigger. The `document_embeddings` table uses an **IVFFLAT** vector index with `lists=100` for efficient cosine similarity search.

System Architecture

FIG 2 — HIGH-LEVEL SYSTEM ARCHITECTURE

Complete Systems Inventory — All Features

Feature / Module	Status	Platform / Technology	Full Description
Frontend SPA	✓ Done	React 18 + TypeScript + Vite + Tailwind CSS + Heroicons; Vercel	Single-page application with role-based routing and protected pages. Persistent single-window layout per role. Dashboard, Rules, Semester, Wiki, Chatbot within one shell component. Responsive UI.
Authentication	✓ Done	Supabase Auth (JWT + email/password)	Email/password login via Supabase Auth. First-login forced password change enforced client side and via `profiles.first_login` flag. Role derived from `profiles.role`; session managed by Supabase JS SDK v2. Visitor state = unauthenticated user.
Database	✓ Done	Supabase PostgreSQL 15 (fully managed)	12 core relational tables with full FK relationships, CHECK constraints, ENUM types, and computed columns. Shared `updated_at` trigger on all tables. pgvector extension enabled for vector similarity search.
Row-Level Security	✓ Done	Supabase RLS Policies (per-table)	Every table has RLS enabled. Review `author_id` is hidden from non-registrars via a dedicated Postgres view. Students see only their own enrollments. Instructors see only their class rosters. Registrar has full access. Public views for dashboard stats.
Auto REST API	✓ Done	Supabase PostgREST (auto-generated)	No custom REST server required. PostgREST auto-exposes all tables and views as typed REST endpoints. Complex operations (GPA calculation, phase transitions) are Postgres stored functions called via `/rpc/name`.
Business Logic Engine	✓ Done	Supabase Edge Functions (Deno + TypeScript)	Stateless serverless functions handle: semester phase transitions with guard logic, course cancellation checks (<3 students), warning issuance, taboo word filtering, GPA recalculation, honor roll eligibility, and notification dispatch. Idempotent where possible.
Realtime Notifications	✓ Done	Supabase Realtime (Websocket channel subscriptions)	Subscribes to `INSERT` events on `warnings`, `enrollments`, and `complaints` tables filtered by user ID. Users receive live toast notifications for new warnings, enrollment confirmations, and complaint resolutions without page refresh.
AI Chatbot (RAG)	✓ Done	pgvector + OpenAI text-embedding-3-small + GPT-4o via Edge Function	College-specific knowledge stored as vector embeddings in `document_embeddings`. On query: embed user message → cosine search top-k chunks → inject into GPT-4o system prompt with role-scoped context → stream response. Fallback with hallucination warning if no local match found.
Taboo Word Filter	✓ Done	Postgres function + Edge Function on review submission	Registrar manages `taboo_words` table. On review submit, Edge Function counts matches. 1+ matches → replace with asterisks in `filtered_body`; base +1 warning for author; ≥3 matches → `is_hidden=true`, issue +2 warnings to author. Original text retained for registrar view only.
Semester Phase Engine	✓ Done	Supabase Edge Function + `semesters.phase` enum	Registrar advances phase via API call. Each transition validates guards (e.g., cannot enter Grading phase if running classes incomplete). Phase stored in `semesters.phase`; UI and API actions gated by reading the current phase value.
Creative: Smart Study Groups	✓ Done	Edge Function + GPT-4o + `study_groups` table	Students request AI-generated study partner suggestions for their enrolled classes. Edge Function collects co-enrolled students with compatible schedules → GPT-4o generates ranked partner suggestions with reasoning → stored in `study_groups` table for future reference.
Deployment	✓ Done	Vercel (frontend SPA) + Supabase Cloud (backend, DB, Auth, Edge Fns)	React SPA deployed on Vercel (GitHub CI/CD integration, free tier). All backend services hosted on Supabase Cloud free tier — sufficient for the ~10-student showcase. No self-hosted servers, containers, or DevOps infrastructure required.

AI Feature Design & Data Flow Diagrams

AI Chatbot — RAG Pipeline (Fig 3)

Semester Phase State Machine (Fig 4)

Warning & Suspension Escalation Reference (Fig 5)

Trigger Event	Target	Immediate Action	Escalation If Unresolved
Enrolled in <2 courses during Running phase	Student	+1 Warning	3 warnings → 1-semester suspension + fine
Class avg rating <2.0	Instructor	+1 Warning	3 warnings → suspended next semester
Grades not submitted by end of Grading period	Instructor	+1 Warning	Same escalation track
Class GPA <2.5 or <2.5 without justification	Instructor	+1 Warning or fired	Registrar discretion; immediate firing possible
All courses cancelled in a semester	Instructor	Suspended next semester	—
Student cumulative GPA <2.0 or same course failed twice	Student	Auto termination	Removed from system permanently
Student GPA 2.0–2.25	Student	+1 Warning + mandatory registrar interview	Suspension if interview not completed
Review contains 1-2 taboo words	Student (author)	+1 Warning; words replaced with *	3 warnings → suspension + fine
Review contains ≥3 taboo words	Student (author)	+2 Warnings; review hidden from system	Same track
Honor Roll achieved (win GPA >3.75 or curve >3.5)	Student	Removes 1 existing warning	—
Premature graduation application (fewer than 8 courses)	Student	+1 Warning	Standard warning track

Technology Stack Summary

Frontend

- **React 18** + TypeScript — component-based SPA, unified layout window
- **Vite** — fast build tool and hot-module replacement dev server
- **Tailwind CSS** — utility-first styling, no custom CSS frameworks
- **Supabase JS SDK v2** — auth, queries, realtime subscriptions
- **React Router v6** — role-gated client-side routing
- **Vercel** — deployment with GitHub CI/CD integration

Backend (Supabase Platform)

- **PostgreSQL 15** — primary database + pgvector extension
- **PostgREST** — auto-generated REST API from DB schema
- **Supabase Auth** — JWT sessions, email/password login
- **Edge Functions (Deno)** — serverless business logic & AI orchestration
- **Realtime** — Websocket subscriptions for live notifications
- **OpenAI API** — GPT-4o (chat) + text-embedding-3-small (RAG)

Design Principle: Supabase consolidates auth, database, API layer, edge compute, realtime, and vector search into one managed platform — eliminating the need for a separate backend server. This keeps operational complexity minimal while fully supporting all spec requirements: AI powered RAG, semester lifecycle automation, role-based data access, and real-time notifications for a showcase-scale (~10 student) deployment.